

# C 语言与计算思维

## 期末复习全攻略

摆渡 (Powered by GitHub Copilot)

2026 年 1 月 7 日

### 使用建议

本复习攻略旨在帮助大家高效梳理《C 语言与计算思维》课程的核心考点。

**建议用法：**请快速浏览全文，对于已经掌握的知识点一带而过，重点关注自己不熟悉或容易混淆的部分（如指针、文件操作、易错点辨析等）。

**祝大家考试顺利，高分通过！**

## 目录

---

|                                   |           |
|-----------------------------------|-----------|
| <b>1 计算机系统与原理</b>                 | <b>3</b>  |
| 1.1 核心考点                          | 3         |
| 1.2 简答题预测                         | 3         |
| <b>2 算法基础</b>                     | <b>4</b>  |
| 2.1 核心考点                          | 4         |
| 2.2 简答题预测                         | 4         |
| <b>3 实验与期末试卷分析</b>                | <b>4</b>  |
| 3.1 实验课核心内容总结                     | 4         |
| 3.2 期末试卷高频考点解析                    | 5         |
| <b>4 C 语言基础语法</b>                 | <b>6</b>  |
| 4.1 数据类型与运算                       | 6         |
| 4.2 输入输出 (Input/Output)           | 7         |
| 4.3 常量与宏定义                        | 8         |
| 4.4 易错点                           | 8         |
| <b>5 控制结构</b>                     | <b>8</b>  |
| <b>6 数组与字符串</b>                   | <b>8</b>  |
| 6.1 基本概念                          | 8         |
| 6.2 常用字符串函数 (<string.h>)          | 9         |
| 6.3 其他重要函数                        | 9         |
| <b>7 指针 (核心难点)</b>                | <b>9</b>  |
| 7.1 指针基础与运算                       | 9         |
| 7.2 指针与数组的深度辨析                    | 10        |
| 7.3 复杂指针声明解析                      | 10        |
| 7.4 简答题预测                         | 10        |
| <b>8 函数与递归</b>                    | <b>11</b> |
| <b>9 结构体、共用体与枚举</b>               | <b>11</b> |
| 9.1 核心考点                          | 11        |
| 9.2 简答题预测                         | 11        |
| <b>10 动态内存管理 (Dynamic Memory)</b> | <b>11</b> |

|                                           |           |
|-------------------------------------------|-----------|
| 10.1 栈 (Stack) 与堆 (Heap) 的区别 . . . . .    | 12        |
| 10.2 核心函数详解 . . . . .                     | 12        |
| 10.3 常见错误与最佳实践 . . . . .                  | 12        |
| <b>11 链表与队列 (Linked List &amp; Queue)</b> | <b>13</b> |
| 11.1 单向链表 (Singly Linked List) . . . . .  | 13        |
| 11.1.1 节点定义 . . . . .                     | 13        |
| 11.1.2 链表的核心操作 . . . . .                  | 13        |
| 11.2 队列 (Queue) . . . . .                 | 14        |
| 11.2.1 队列的结构体定义 (链式队列) . . . . .          | 14        |
| 11.2.2 入队与出队 . . . . .                    | 15        |
| 11.3 易错点 . . . . .                        | 15        |
| <b>12 文件操作详解 (File I/O)</b>               | <b>15</b> |
| 12.1 文件打开与关闭 . . . . .                    | 16        |
| 12.2 文件打开模式 (Mode) 深度解析 . . . . .         | 16        |
| 12.3 读写函数辨析 . . . . .                     | 17        |
| 12.4 文件定位与随机读写 . . . . .                  | 17        |
| 12.5 文件操作常见考点与易错点 . . . . .               | 18        |
| 12.6 预处理指令补充 . . . . .                    | 19        |
| <b>13 核心算法详解与易错点深度剖析</b>                  | <b>19</b> |
| 13.1 排序算法 (Sorting) . . . . .             | 19        |
| 13.1.1 冒泡排序 (Bubble Sort) . . . . .       | 19        |
| 13.1.2 选择排序 (Selection Sort) . . . . .    | 20        |
| 13.1.3 插入排序 (Insertion Sort) . . . . .    | 21        |
| 13.2 查找算法 (Searching) . . . . .           | 21        |
| 13.2.1 二分查找 (Binary Search) . . . . .     | 21        |
| 13.3 数学与逻辑算法 . . . . .                    | 22        |
| 13.3.1 辗转相除法 (GCD) . . . . .              | 22        |
| 13.3.2 素数判断 (Prime Check) . . . . .       | 22        |
| 13.3.3 常见数学公式实现 . . . . .                 | 23        |
| 13.4 字符串处理算法 . . . . .                    | 24        |
| 13.4.1 字符串逆序 . . . . .                    | 24        |
| 13.5 考试手写代码“救命”技巧 . . . . .               | 24        |
| <b>14 C 语言易混淆概念辨析</b>                     | <b>24</b> |
| <b>15 冷门与进阶考点拾遗</b>                       | <b>25</b> |

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>16 全卷易错点与陷阱大盘点</b>                             | <b>26</b> |
| 16.1 语法与逻辑陷阱 . . . . .                            | 26        |
| 16.2 数组与指针陷阱 . . . . .                            | 27        |
| 16.3 控制结构与输入输出陷阱 . . . . .                        | 28        |
| 16.4 函数参数传递陷阱 . . . . .                           | 29        |
| <b>17 简答题预测汇总</b>                                 | <b>30</b> |
| 17.1 计算机基础与算法 . . . . .                           | 30        |
| 17.2 C 语言核心概念 . . . . .                           | 30        |
| 17.3 常见算法特点与比较 . . . . .                          | 31        |
| 17.4 实验课补充简答题 . . . . .                           | 32        |
| <b>A 冯·诺依曼体系结构详解</b>                              | <b>33</b> |
| <b>B 附录 1：程序填空题常考模式代码详解</b>                       | <b>35</b> |
| B.1 数学公式实现 . . . . .                              | 35        |
| B.2 最大公约数 (辗转相除法) . . . . .                       | 35        |
| B.3 条件判断 (三目运算符嵌套) . . . . .                      | 35        |
| <b>C 附录 2：标准算法模板</b>                              | <b>36</b> |
| C.1 排序算法 . . . . .                                | 36        |
| C.2 查找算法 . . . . .                                | 37        |
| C.3 其他常用算法 . . . . .                              | 37        |
| <b>D 附录 3：复杂指针类型深度解析</b>                          | <b>37</b> |
| D.1 1. const 与指针的爱恨情仇 . . . . .                   | 38        |
| D.2 2. 指针数组 vs 数组指针 . . . . .                     | 38        |
| D.3 3. 函数指针与回调机制 . . . . .                        | 39        |
| D.4 4. 二维数组 vs 二维动态数组 (内存模型与访问) . . . . .         | 40        |
| D.5 5. 真题解析：二维数组指针表示法 (a[i][j] 的 N 种写法) . . . . . | 41        |
| D.6 1. 转义字符 (Escape Characters) 全解 . . . . .      | 42        |
| D.7 2. 数值转义：\ddd vs \xhh . . . . .                | 42        |

# 1 计算机系统与原理

---

## 1.1 核心考点

- **图灵测试 (Turing Test):** 测试机器是否具有人类智能的标准。如果测试者在不接触被测试者的情况下, 无法区分人和机器, 则认为机器通过测试。
- **计算机系统组成:**
  - **硬件 (Hardware):** CPU、存储器、输入/输出设备 (冯·诺依曼结构核心)。
  - **软件 (Software):** 系统软件 (OS、驱动)、应用软件 (Office、游戏)。
- **冯·诺依曼体系结构:**
  - **核心思想: 存储程序 (Stored Program)。** 程序和数据都以二进制形式存储在存储器中。
  - **五大部件:** 运算器、控制器、存储器、输入设备、输出设备。 ([点击查看详细原理与图解](#))
- **数据表示:**
  - **二进制:** 计算机内部使用二进制 (0 和 1)。
  - **原码、反码与补码:**
    - \* **原码:** 最高位为符号位 (0 正 1 负), 其余位表示数值大小。
    - \* **反码:** 正数的反码同原码; 负数的反码为原码符号位不变, 其余位按位取反。
    - \* **补码:** 正数的补码同原码; 负数的补码为反码加 1。
    - \* **存储规则:** 负数在计算机中以补码形式存储。
    - \* **示例 (以 8 位二进制数 -5 为例):**
      - 原码: 1000 0101 (最高位 1 表示负, 数值 5 为 101)
      - 反码: 1111 1010 (符号位不变, 数值位取反)
      - 补码: 1111 1011 (反码 + 1)

## 1.2 简答题预测

**Q: 什么是“存储程序”原理?**

A: 将程序 (指令序列) 和数据一起存储在计算机的存储器中, 计算机在运行时自动地从存储器中取出指令并执行, 而不需要人工干预。

**Q: 简述硬件与软件的关系。**

A: 硬件是计算机的躯壳 (物理实体), 软件是计算机的灵魂 (思想)。硬件提供物质基础, 软件指挥硬件工作。两者相辅相成, 缺一不可。

## 2 算法基础

---

### 2.1 核心考点

- 定义：解决特定问题的一系列有限步骤。
- 五大特性：
  1. 有穷性：步骤有限。
  2. 确定性：指令清晰无歧义。
  3. 输入：0 个或多个。
  4. 输出：1 个或多个。
  5. 可行性：每一步都能通过基本运算实现。

### 2.2 简答题预测

Q: 什么是算法？它有哪些基本特征？

A: 算法是解决特定问题的一系列明确、有限的指令。基本特征包括：有穷性、确定性、可行性、输入、输出。

## 3 实验与期末试卷分析

---

### 3.1 实验课核心内容总结

- 基础运算与日期：
  - 模拟计算器：注意运算符优先级和除零错误。
  - 日期计算：闰年判断  $(year \% 4 == 0 \ \&\& \ year \% 100 != 0) \ || \ (year \% 400 == 0)$ 。
- 数组与结构体：
  - 有序数组 vs 无序数组：插入、删除、查找的时间复杂度对比。
  - 结构体数组：如客户信息管理、歌唱比赛评分（去掉最高最低分求平均）。
- 递归应用：
  - 汉诺塔：经典的递归分治问题。
  - 预测赢家：博弈论中的递归求解。
- 大数运算：
  - 使用字符数组或整型数组模拟超大整数的加法，注意进位处理。
- 栈的应用：
  - JSON 字符串括号匹配：利用栈的后进先出特性，遇到左括号入栈，遇到右括号出栈匹配。

## 3.2 期末试卷高频考点解析

- 选择题陷阱与对比辨析：

- 字符串长度：strlen 遇到 \0 停止，转义字符（如 \101）算一个字符。

- \* 对比：sizeof 统计内存空间大小（含 \0）；strlen 统计有效字符长度（不含 \0）。

- \* 例子：

```
1 char s[] = "hi";
2 // sizeof(s) -> 3
3 // strlen(s) -> 2
```

- 逻辑短路：a || b++，若 a 为真，b++ 不执行。

- \* 对比：&& 左假则右不执行；|| 左真则右不执行。位运算 & 和 | 无此特性。

- \* 例子：

```
1 int a = 1, b = 0;
2 if (a || b++) { ... }
3 // 执行后 b 仍为 0
```

- \* 真题陷阱：

```
1 int a = 1, b = 2, m = 0, n = 0, k;
2 k = (n = b > a) || (m = a < b);
3 // 解析：
4 // 1. b > a (2 > 1) 为真(1)，赋给 n, n 变为 1。
5 // 2. 左侧表达式 (n=...) 结果为 1 (真)。
6 // 3. || 发生短路，右侧 (m=a<b) 不执行！m 保持 0。
7 // 4. k 得到 || 的结果 1。
```

- 指针类型：区分 int \*p[10] (指针数组) 和 int (\*p)[10] (数组指针)。

- \* 对比：前者是数组（存指针）；后者是指针（指数组）。

- \* 例子：

```
1 // 指针数组
2 char *strs[3] = {"A", "B", "C"};
3 // 数组指针
4 int arr[10];
5 int (*p)[10] = &arr;
```

- 作用域：局部变量屏蔽同名全局变量。

- \* 辨析：同名时，谁近用谁（局部优先）。

- \* 例子：

```
1 int x = 10;
2 void f() {
3     int x = 5;
4     printf("%d", x); // 输出 5
5 }
```

- 程序填空题常考模式:

- 详见附录 [程序填空题常考模式代码详解](#)。

- 代码阅读题:

- 能够正确跟踪指针的移动和数组元素的访问。

- 理解 `sizeof` 在不同情况下的值 (数组名 vs 指针)。

- \* 对比: 数组名在定义域内 `sizeof` 为总字节数; 传参后退化为指针, `sizeof` 为指针大小 (4 或 8)。

- \* 例子:

```
1 int a[10]; // sizeof(a) -> 40
2 void f(int a[]) {
3     // sizeof(a) -> 4/8 (指针)
4 }
```

## 4 C 语言基础语法

### 4.1 数据类型与运算

- 数据类型大小: `sizeof` 是运算符。

- `char` (1), `short` (2), `int` (4), `long` (4 或 8), `double` (8)。

- 类型转换:

- 隐式: `int + double`  $\rightarrow$  `double`。

- 强制: `(int)(3.14 + 0.9)` 结果是 4。

- 自增自减:

- `b = a++`: 先赋值, 后自增。

- `b = ++a`: 先自增, 后赋值。

- 真题陷阱: `k=(n=b>a)||(m=a<b)`。注意逻辑短路, 若前半部分为真, 后半部分不执行。

- 位运算:

- `&`, `|`, `^`, `~`, `<<`, `>>`。

- 技巧: `a << 1` (乘 2), `a >> 1` (除 2), `n & 1` (判断奇偶)。



- 运算符优先级（由高到低）：

1. 初等：() [] -> .
2. 单目：! ~ ++ -- sizeof (type) \* & (右到左)
3. 算术：\* / % → + -
4. 移位：<< >>
5. 关系 (大小)：> < >= <=
6. 关系 (相等)：== != (优先级低于大小比较)
7. 位逻辑：& → ^ → |
8. 逻辑：&& → ||
9. 三目：?: (右到左)
10. 赋值：= += -= ... (右到左)
11. 逗号：,

#### 优先级口诀与陷阱

口诀：单目 > 算术 > 关系 > 逻辑 > 三目 > 赋值

注意 1：关系运算符中，大小比较 (> <) 高于 相等比较 (== !=)。

例如：a > b == c 等价于 (a > b) == c。

注意 2：右到左结合性（单目、三目、赋值）。

赋值：a = b = c = 0 → a = (b = (c = 0))。

单目：\*p++ → \*(p++)（先取值后自增，再解引用原地址）。

## 4.2 输入输出 (Input/Output)

- 格式化输出 (printf)：

- %d (整型), %f (浮点), %c (字符), %s (字符串), %p (指针地址)。
- 精度控制：%.2f (保留 2 位小数), %5d (宽 5 位右对齐), %-5d (宽 5 位左对齐)。

- 格式化输入 (scanf)：

- 地址符：除了字符串数组名外，变量前必须加 &。
- 分隔符：若格式串为 "%d,%d"，输入时必须加逗号。
- 缓冲区残留：混合输入字符和数字时，回车符可能被 %c 读取。建议在 %c 前加空格：scanf(" %c", &ch);。

- 字符输入输出：

- getchar() / putchar()：一次只处理一个字符。

### 4.3 常量与宏定义

- **const 常量:**
  - `const int MAX = 100;`
  - 有类型检查, 占用内存 (通常在只读区)。
- **宏定义 (#define):**
  - `#define MAX 100`
  - 预处理阶段文本替换, 无类型检查, 不占内存。
  - 注意: 宏定义末尾不要加分号。
  - 真题陷阱 (带参宏): 纯文本替换, 不自动加括号。
    - \* 错误: `#define S(x) x*x`  $\rightarrow$  `S(a+b)` 变为 `a+b*a+b`。
    - \* 正确: `#define S(x) ((x)*(x))`  $\rightarrow$  `((a+b)*(a+b))`。

### 4.4 易错点

- 除法陷阱: `5 / 2 = 2`, `5.0 / 2 = 2.5`。
- 逻辑短路: `A || B` (A 真 B 不执行), `A && B` (A 假 B 不执行)。
- 浮点比较: 严禁 `f == 0.0`, 应用 `fabs(f) < 1e-6`。
- 赋值与相等: `if(a=2)` 永远为真, 且 `a` 变为 2。这是极高频考点。

## 5 控制结构

---

- **Switch-Case:** `case` 后必须是整型或字符型常量。注意**穿透性** (漏写 `break`)。
- 循环: `continue` 跳过本次剩余, `break` 跳出循环。
- 易错点: 悬挂 `else` (与最近的 `if` 匹配); 分号错误 (`if(x); ...`)。
- 真题模式: 利用 `do...while` 计算泰勒级数 (如 `sin(x)`), 注意循环终止条件通常是 `fabs(term) < 1e-5`。

## 6 数组与字符串

---

### 6.1 基本概念

- 定义: 字符串是以空字符 `'\0'` 结尾的字符数组。
- 初始化:
  - `char s[] = "hello";` (自动加 `'\0'`, 长度为 6, 内容可变)
  - `char *p = "hello";` (字符串常量, 通常存储在只读数据区, 不可修改)

- **sizeof vs strlen:**

- **sizeof:** 计算数组占用的总字节数 (含 '\0')。
- **strlen:** 计算字符串的有效长度 (不含 '\0')。
- **真题陷阱:** `strlen("abc\0def")` 结果是 3 (遇到第一个 \0 停止)。

## 6.2 常用字符串函数 (<string.h>)

- **复制 (Copy):**

- `strcpy(dest, src)`: 将 `src` 复制到 `dest`。
- **危险:** 若 `dest` 空间不足, 会发生缓冲区溢出。
- `strncpy(dest, src, n)`: 最多复制 `n` 个字符。注意: 若 `src` 长度  $> n$ , 则 `dest` 不会自动添加 '\0', 需手动添加。

- **拼接 (Concatenate):**

- `strcat(dest, src)`: 将 `src` 拼接到 `dest` 后面。
- **要求:** `dest` 必须有足够的剩余空间, 且必须以 '\0' 结尾。
- `strncat(dest, src, n)`: 拼接前 `n` 个字符, 会自动添加 '\0'。

- **比较 (Compare):**

- `strcmp(s1, s2)`: 按字典序比较。
- **返回值:** 0 (相等),  $>0$  (`s1` 大),  $<0$  (`s1` 小)。
- **陷阱:** 严禁使用 `if(s1 == s2)` 比较内容 (这是比较地址!)

- **查找 (Search):**

- `strchr(s, c)`: 查找字符 `c` 首次出现的位置 (返回指针)。
- `strstr(s1, s2)`: 查找子串 `s2` 首次出现的位置。

## 6.3 其他重要函数

- `sprintf(buf, "%d", 123)`: 将数据格式化输出到字符串中 (常用于数字转字符串)。
- `atoi(s)` / `atof(s)`: 字符串转整数/浮点数 (<stdlib.h>).

## 7 指针 (核心难点)

---

### 7.1 指针基础与运算

- **定义与解引用:**

- `int *p = &a;`: `p` 存储 `a` 的地址。

- `*p`: 访问 `p` 指向的内存单元 (即 `a`)。
- 口诀: `&` 取地址, `*` 取值。
- 指针运算:
  - `p + 1`: 不是地址加 1, 而是地址增加 `sizeof(类型)`。
  - `p1 - p2`: 返回两指针之间的元素个数 (不是字节数)。
  - 示例: 若 `int a[5]`, `p=a`, 则 `*(p+2)` 等价于 `a[2]`。

## 7.2 指针与数组的深度辨析

- 访问方式等价性:
  - $a[i] \iff *(a+i) \iff *(p+i) \iff p[i]$ 。
- 本质区别:
  - 数组名 `a` 是常量指针, 不可修改 (`a++` 非法)。
  - 指针变量 `p` 是变量, 可以修改指向 (`p++` 合法)。
- 二维数组指针:
  - `int a[3][4]`;
  - `a`: 指向首行 (类型 `int (*)[4]`)。
  - `a[0]`: 指向首行首元素 (类型 `int *`)。
  - $a[i][j] \iff *((a+i)+j)$ 。

## 7.3 复杂指针声明解析

- `const` 修饰 (“左定值, 右定向”):
  - `const int *p: *p` (内容) 不可变, `p` (指向) 可变。
  - `int * const p: p` (指向) 不可变, `*p` (内容) 可变。
- 指针数组 vs 数组指针:
  - `int *p[10]`: 数组, 存了 10 个指针。
  - `int (*p)[10]`: 指针, 指向长度为 10 的数组。
- 函数指针:
  - 定义: `int (*func)(int, int);`
  - 用途: 回调函数 (如 `qsort` 的比较函数)。

## 7.4 简答题预测

Q: 什么是指针? 为什么要使用指针?

A: 指针是存储内存地址的变量。作用: 1. 直接操作内存; 2. 函数间传递大数据

(避免拷贝); 3. 动态内存分配; 4. 操作复杂数据结构。

## 8 函数与递归

---

- 传值 vs 传址: 修改实参需传指针。数组传参退化为指针。
- 递归 (Recursion):
  - 要素: 基准情况 (Base Case) + 递归步骤 (Recursive Step)。
  - 优缺点: 代码简洁但开销大 (栈空间)。
- 存储类别:
  - auto: 栈内存。
  - static: 延长局部变量生命周期; 限制全局变量作用域。
  - 真题考点: 局部变量与全局变量同名时, 局部变量优先 (屏蔽全局变量)。

## 9 结构体、共用体与枚举

---

### 9.1 核心考点

- 结构体 (Struct):
  - 成员拥有独立内存。
  - 内存对齐: 总大小  $\geq$  成员之和。
  - 真题考点: `typedef struct {...} Name;` 定义别名, 方便使用。
- 共用体 (Union):
  - 成员共享同一块内存。
  - 总大小 = 最大成员大小。
- 枚举 (Enum): 默认从 0 开始递增。

### 9.2 简答题预测

Q: 简述结构体和共用体的区别。

A: 1. 内存: 结构体成员独立, 共用体成员共享。2. 大小: 结构体累加 (含对齐), 共用体取最大。3. 赋值: 共用体同一时间只能存一个值。

## 10 动态内存管理 (Dynamic Memory)

---

## 10.1 栈 (Stack) 与堆 (Heap) 的区别

- 栈 (Stack):
  - 由编译器自动分配释放。
  - 存放局部变量、函数参数。
  - 空间较小，速度快。
- 堆 (Heap):
  - 由程序员手动分配 (malloc) 和释放 (free)。
  - 空间大，但容易产生内存泄漏或碎片。

## 10.2 核心函数详解

- malloc: `void *malloc(size_t size);`
  - 分配指定字节数的内存。
  - 内容未初始化 (随机值)。
  - 返回 `void*`, 通常需要强制类型转换 (C 语言中可省略, 但建议写上)。
  - 示例: `int *p = (int*)malloc(n * sizeof(int));`
- calloc: `void *calloc(size_t n, size_t size);`
  - 分配  $n \times size$  字节。
  - 自动初始化为 0。
- realloc: `void *realloc(void *ptr, size_t new_size);`
  - 调整已分配内存的大小。
  - 可能移动内存块位置, 必须接收返回值更新指针。
- free: `void free(void *ptr);`
  - 释放内存, 归还给系统。
  - 注意: 释放后指针变量本身的价值未变 (悬空指针), 建议立即置为 `NULL`。

## 10.3 常见错误与最佳实践

- 忘记检查 `NULL`:

```
1  int *p = (int*)malloc(1000 * sizeof(int));
2  if (p == NULL) {
3      // 处理分配失败
4      exit(1);
5  }
6
```

- 内存泄漏 (Memory Leak): 申请了内存但没有 free, 导致程序占用内存越来越多。
- 悬空指针 (Dangling Pointer): 使用已经 free 过的指针。
- 重复释放: 对同一个指针 free 两次 (会导致崩溃)。

## 11 链表与队列 (Linked List & Queue)

### 11.1 单向链表 (Singly Linked List)

链表是一种物理存储单元上非连续、非顺序的存储结构, 数据元素的逻辑顺序是通过链表中的指针链接次序实现的。

#### 11.1.1 节点定义

```
1 typedef struct Node {  
2     int data;           // 数据域  
3     struct Node *next;  // 指针域, 指向下一个节点  
4 } Node;
```

#### 11.1.2 链表的核心操作

##### 1. 创建新节点

```
1 Node* createNode(int value) {  
2     Node* newNode = (Node*)malloc(sizeof(Node));  
3     if (newNode == NULL) {  
4         printf("Memory allocation failed!\n");  
5         exit(1);  
6     }  
7     newNode->data = value;  
8     newNode->next = NULL;  
9     return newNode;  
10 }  
11
```

##### 2. 头插法 (Insert at Head)

```
1 void insertHead(Node** head, int value) {  
2     Node* newNode = createNode(value);  
3     newNode->next = *head; // 新节点的 next 指向当前头节点  
4     *head = newNode;      // 更新头指针指向新节点  
5 }  
6
```

### 3. 尾插法 (Insert at Tail)

```
1 void insertTail(Node** head, int value) {
2     Node* newNode = createNode(value);
3     if (*head == NULL) {
4         *head = newNode;
5         return;
6     }
7     Node* temp = *head;
8     while (temp->next != NULL) { // 遍历到最后一个节点
9         temp = temp->next;
10    }
11    temp->next = newNode;
12 }
13
```

### 4. 链表遍历

```
1 void printList(Node* head) {
2     Node* current = head;
3     while (current != NULL) {
4         printf("%d -> ", current->data);
5         current = current->next;
6     }
7     printf("NULL\n");
8 }
9
```

## 11.2 队列 (Queue)

队列是一种特殊的线性表，只允许在表的前端（front）进行删除操作，在表的后端（rear）进行插入操作。遵循 **FIFO (First In First Out, 先进先出)** 原则。

### 11.2.1 队列的结构体定义 (链式队列)

使用链表实现队列可以避免数组大小固定的限制。

```
1 typedef struct {
2     Node *front; // 队头指针，用于出队
3     Node *rear;  // 队尾指针，用于入队
4 } Queue;
5
6 // 初始化队列
7 void initQueue(Queue *q) {
8     q->front = q->rear = NULL;
9 }
```



### 11.2.2 入队与出队

- 入队 (Enqueue): 在 rear 后添加新节点。

```
1 void enqueue(Queue *q, int value) {  
2     Node* newNode = createNode(value);  
3     if (q->rear == NULL) { // 队列为空  
4         q->front = q->rear = newNode;  
5         return;  
6     }  
7     q->rear->next = newNode; // 旧队尾指向新节点  
8     q->rear = newNode;      // 更新队尾指针  
9 }  
10
```

- 出队 (Dequeue): 移除 front 指向的节点。

```
1 int dequeue(Queue *q) {  
2     if (q->front == NULL) return -1; // 队列为空  
3  
4     Node *temp = q->front;  
5     int data = temp->data;  
6  
7     q->front = q->front->next; // 队头指针后移  
8  
9     if (q->front == NULL) { // 如果出队后队列变空, rear也要置空  
10        q->rear = NULL;  
11    }  
12  
13    free(temp); // 释放节点内存  
14    return data;  
15 }  
16
```

### 11.3 易错点

- 内存泄漏: 删除节点或销毁链表时, 记得 free 内存。
- 空链表处理: 在插入、删除操作时, 要考虑头指针为 NULL 的情况。
- 断链: 插入节点时, 先连后断。

## 12 文件操作详解 (File I/O)

## 12.1 文件打开与关闭

- 打开文件: `FILE *fp = fopen("filename", "mode");`
- 检查是否成功: 必须检查 `fp` 是否为 `NULL`。

```
1     if ((fp = fopen("data.txt", "r")) == NULL) {  
2         printf("Cannot open file.\n");  
3         exit(1);  
4     }  
5
```

- 关闭文件: `fclose(fp);` 也就是保存文件, 释放缓冲区。

## 12.2 文件打开模式 (Mode) 深度解析

- 文本模式 (Text Mode):
  - "r" (read): 只读。
    - \* 文件必须存在, 否则返回 `NULL`。
    - \* 指针指向文件开头。
  - "w" (write): 只写。
    - \* 文件不存在则创建。
    - \* 文件存在则清空 (Truncate to zero length) ——极度危险, 慎用!
    - \* 指针指向文件开头。
  - "a" (append): 追加。
    - \* 文件不存在则创建。
    - \* 文件存在则保留原内容, 指针指向文件末尾。
    - \* 只能在末尾写, 不能修改中间内容。
  - "r+": 读写 (覆盖模式)。
    - \* 文件必须存在。
    - \* 指针指向开头。可以读, 也可以从头开始覆盖写。
  - "w+": 读写 (清空模式)。
    - \* 文件不存在则创建, 存在则清空。
    - \* 既能读也能写, 但刚打开时文件是空的。
  - "a+": 读写追加。
    - \* 文件不存在则创建, 存在则保留。
    - \* 读: 指针默认在开头 (可以读旧数据)。
    - \* 写: 永远在末尾追加 (无论指针在哪)。
- 二进制模式 (Binary Mode):

- 标志：在模式后加 **b**，如 `"rb"`, `"wb"`, `"ab+"`。
- 核心区别：换行符处理。
  - \* 文本模式：在 Windows 下，写入 `\n` 会自动转换为 `\r\n` (CRLF)；读取 `\r\n` 会自动转换为 `\n`。Linux 下无此转换。
  - \* 二进制模式：不做任何转换，读写原始字节。
- 适用场景：图片、音频、视频、结构体数据块 (`fread/fwrite`) 必须用二进制模式，否则数据会损坏。

## 12.3 读写函数辨析

- 字符读写：
  - `ch = fgetc(fp)`：读一个字符。结束返回 `EOF`。
  - `fputc(ch, fp)`：写一个字符。
- 字符串读写：
  - `fgets(str, n, fp)`：读取一行（最多  $n - 1$  个字符），保留换行符，自动加 `\0`。
  - `fputs(str, fp)`：写入字符串，不会自动加换行符。
- 格式化读写（文本文件）：
  - `fscanf(fp, "%d", &num)`：从文件读。
  - `fprintf(fp, "Result: %d", num)`：写入文件。
- 数据块读写（二进制文件）：
  - `fread(buffer, size, count, fp)`：从文件读 `count` 个 `size` 大小的块到 `buffer`。
  - `fwrite(buffer, size, count, fp)`：将 `buffer` 中 `count` 个 `size` 大小的块写入文件。
  - 返回值：成功读/写的块数 (`count`)。

## 12.4 文件定位与随机读写

- `rewind(fp)`：文件指针回到开头。
- `ftell(fp)`：返回当前文件指针位置（字节偏移量）。
- `fseek(fp, offset, origin)`：移动指针。
  - `SEEK_SET (0)`：文件头。
  - `SEEK_CUR (1)`：当前位置。
  - `SEEK_END (2)`：文件尾。

## 12.5 文件操作常见考点与易错点

- **feof 的误用 (经典考点):**

- 错误写法: `while(!feof(fp))`。
- 原因: `feof` 只有在读取操作尝试读取文件尾之后才会返回真, 而不是到达文件尾就立即返回真。这会导致循环多执行一次 (最后一次读取失败但循环体仍执行)。
- 正确写法: 利用读取函数的返回值作为循环条件。

```
1 // 文本读整数
2 while (fscanf(fp, "%d", &n) == 1) { ... }
3 // 文本读行
4 while (fgets(buf, 100, fp) != NULL) { ... }
5 // 二进制读块
6 while (fread(&data, sizeof(data), 1, fp) == 1) { ... }
7
```

- **文本文件 vs 二进制文件存储大小:**

- 考题常问: 整数 12345 存入文件占用多少字节?
- 文本模式: 按字符存, 占用 **5 个字节** ('1','2','3','4','5')。
- 二进制模式: 按内存原样存 (int), 通常占用 **4 个字节**。

- **标准流 (Standard Streams):**

- 系统自动打开的三个文件指针:
- `stdin` (标准输入, 对应键盘)
- `stdout` (标准输出, 对应屏幕)
- `stderr` (标准错误, 对应屏幕, 无缓冲)
- 技巧: `fprintf(stdout, ...)` 等价于 `printf(...)`。

- **文件打开失败检测:**

- 必须检查 `fopen` 的返回值是否为 `NULL`。

```
1 FILE *fp = fopen("data.txt", "r");
2 if (fp == NULL) {
3     perror("File open error"); // 输出错误原因
4     return -1;
5 }
6
```

- **读写同步 (Update Mode):**

- 在 `r+`, `w+` 等模式下, 读与写操作之间必须插入 `fseek`, `fsetpos` 或 `fflush`, 否则文件指针位置会混乱导致数据错误。

- **路径分隔符:**

- Windows 路径 C:\test.txt 在 C 字符串中必须转义: "C:\\test.txt" 或使用正斜杠 "C:/test.txt"。

## 12.6 预处理指令补充

- Include: <> 查系统库, "" 查当前目录。

# 13 核心算法详解与易错点深度剖析

## 13.1 排序算法 (Sorting)

### 13.1.1 冒泡排序 (Bubble Sort)

**核心思想：**像水底的气泡一样，每一轮通过两两比较和交换，将当前未排序部分的最大值“浮”到数组的最右端。

```
1 void bubble_sort(int arr[], int n) {
2     int i, j, temp;
3     // 外层循环：控制排序轮数，共需 n-1 轮
4     for (i = 0; i < n - 1; i++) {
5         // 内层循环：负责两两比较
6         // 易错点1：边界是 n-1-i，因为最后 i 个元素已经排好序了
7         for (j = 0; j < n - 1 - i; j++) {
8             if (arr[j] > arr[j + 1]) { // 升序用 >，降序用 <
9                 temp = arr[j];
10                arr[j] = arr[j + 1];
11                arr[j + 1] = temp;
12            }
13        }
14    }
15 }
```

**易错点与考点：**

- 内层循环边界:  $j < n - 1 - i$ 。
  - 如果写成  $j < n - 1$ ，程序也能跑，但效率低。
  - 如果写成  $j < n$ ，在访问  $arr[j+1]$  时会数组越界。
- 交换位置：交换是在内层循环的 if 语句中发生的。
- 稳定性：冒泡排序是稳定的。
- 时间复杂度：平均和最坏情况均为  $O(n^2)$ 。

- 推导：外层循环执行  $n - 1$  次。内层循环第 1 轮比较  $n - 1$  次，第 2 轮比较  $n - 2$  次……直到最后 1 次。
- 总比较次数 =  $(n - 1) + (n - 2) + \cdots + 1 = \frac{n(n-1)}{2}$ ，属于  $O(n^2)$  级别。

### 13.1.2 选择排序 (Selection Sort)

**核心思想：**每一轮从未排序区域中找到最小值的下标，然后将其与未排序区域的第一个元素交换。

```
1 void selection_sort(int arr[], int n) {
2     int i, j, min_idx, temp;
3     // 外层循环：控制当前要确定的位置，从 0 到 n-2
4     for (i = 0; i < n - 1; i++) {
5         min_idx = i; // 假设当前位置 i 就是最小值
6
7         // 内层循环：在剩余元素中找真正最小值下标
8         for (j = i + 1; j < n; j++) {
9             if (arr[j] < arr[min_idx]) {
10                 min_idx = j; // 更新最小值下标，注意这里不交换！
11             }
12         }
13
14         // 易错点2：交换发生在内层循环结束后
15         if (min_idx != i) {
16             temp = arr[i];
17             arr[i] = arr[min_idx];
18             arr[min_idx] = temp;
19         }
20     }
21 }
```

**易错点与考点：**

- 交换时机：千万不要在内层循环里交换！内层循环只负责找下标 (min\_idx)。
- 比较对象：内层循环是 `if (arr[j] < arr[min_idx])`，而不是 `arr[j] < arr[i]`。
- 稳定性：选择排序是不稳定的。
- 时间复杂度：无论数组是否有序，均为  $O(n^2)$ 。
  - 推导：无论数据如何，内层循环的比较次数固定。
  - 第 1 轮找最小值比较  $n - 1$  次，第 2 轮比较  $n - 2$  次……总次数恒为  $\frac{n(n-1)}{2}$ 。

### 13.1.3 插入排序 (Insertion Sort)

**核心思想：**像打扑克牌理牌一样，将新抓到的牌（key）插入到左手已经排好序的牌堆中的正确位置。

```
1 void insertion_sort(int arr[], int n) {
2     int i, j, key;
3     // 从第2个元素开始（下标1），因为第1个元素默认有序
4     for (i = 1; i < n; i++) {
5         key = arr[i]; // 待插入的牌
6         j = i - 1;    // 左边已排序区域的最后一个元素
7
8         // 易错点3: while 循环条件
9         // j >= 0 保证不越界
10        // arr[j] > key 保证找到比 key 大的元素往后挪
11        while (j >= 0 && arr[j] > key) {
12            arr[j + 1] = arr[j]; // 元素后移
13            j--; // 继续向前找
14        }
15        arr[j + 1] = key; // 插入到正确位置
16    }
17 }
```

**易错点与考点：**

- 后移操作：arr[j + 1] = arr[j] 是覆盖操作，所以必须先保存 key。
- 插入位置：循环结束时，key 应该放在 j + 1 的位置。
- 稳定性：插入排序是稳定的。
- 时间复杂度：
  - 最坏情况 ( $O(n^2)$ )：数组完全逆序。插入第  $i$  个元素时，需要向前比较  $i$  次。总次数  $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$ 。
  - 最好情况 ( $O(n)$ )：数组已有序。内层 while 循环条件一开始就不满足 (arr[j] < key)，每次只比较 1 次，不移动。总共比较  $n - 1$  次。

## 13.2 查找算法 (Searching)

### 13.2.1 二分查找 (Binary Search)

**前提条件：**数组必须是有序的。

```
1 int binary_search(int arr[], int n, int key) {
2     int low = 0, high = n - 1;
3     int mid;
4 }
```

```
5 while (low <= high) { // 易错点4: 是 <= 不是 <
6     // 易错点5: 防溢出写法
7     mid = low + (high - low) / 2;
8
9     if (arr[mid] == key) {
10         return mid; // 找到了
11     } else if (arr[mid] < key) {
12         low = mid + 1; // 在右边, low 必须 +1
13     } else {
14         high = mid - 1; // 在左边, high 必须 -1
15     }
16 }
17 return -1; // 没找到
18 }
```

#### 易错点与考点:

- 循环条件: `while (low <= high)`。如果写成 `<`, 会漏掉边界情况。
- 死循环陷阱: 更新边界时必须是 `mid + 1` 或 `mid - 1`。
- Mid 计算: `(low + high) / 2` 可能溢出, 推荐 `low + (high - low) / 2`。

## 13.3 数学与逻辑算法

### 13.3.1 辗转相除法 (GCD)

核心思想:  $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ 。

```
1 // 递归版
2 int gcd(int a, int b) {
3     if (b == 0) return a;
4     return gcd(b, a % b);
5 }
```

### 13.3.2 素数判断 (Prime Check)

核心思想: 除了 1 和它本身外, 不能被其他数整除。

```
1 #include <math.h> // 需要用到 sqrt
2
3 int is_prime(int n) {
4     if (n <= 1) return 0; // 易错点6: 1 不是素数
5
6     // 优化: 只需要判断到 sqrt(n)
7     int limit = (int)sqrt(n);
```



```
8     for (int i = 2; i <= limit; i++) {
9         if (n % i == 0) {
10             return 0; // 发现因子，不是素数
11         }
12     }
13     return 1; // 是素数
14 }
```

易错点与考点：

- 特判 1：1 既不是质数也不是合数，必须单独判断返回 0。
- 循环终止条件： $i * i \leq n$  或  $i \leq \text{sqrt}(n)$ 。

### 13.3.3 常见数学公式实现

- 海伦公式 (Heron's Formula)：已知三角形三边  $a, b, c$ ，求面积。

$$p = \frac{a + b + c}{2}, \quad S = \sqrt{p(p-a)(p-b)(p-c)}$$

```
1 #include <math.h>
2 double area(double a, double b, double c) {
3     double p = (a + b + c) / 2.0;
4     return sqrt(p * (p - a) * (p - b) * (p - c));
5 }
6
```

- 泰勒级数求  $\sin(x)$ ： $\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$

```
1 #include <math.h>
2 double my_sin(double x) {
3     double sum = 0, term = x;
4     int n = 1;
5     while (fabs(term) >= 1e-5) { // 精度控制
6         sum += term;
7         term = -term * x * x / ((n + 1) * (n + 2)); // 递推公式
8         n += 2;
9     }
10    return sum;
11 }
12
```

## 13.4 字符串处理算法

### 13.4.1 字符串逆序

核心思想：双指针法，首尾交换，向中间逼近。

```
1 void reverse_string(char str[]) {
2     int len = strlen(str);
3     int i = 0;           // 头指针
4     int j = len - 1;     // 尾指针
5     char temp;
6
7     while (i < j) {
8         temp = str[i];
9         str[i] = str[j];
10        str[j] = temp;
11        i++;
12        j--;
13    }
14 }
```

易错点与考点：

- 尾指针位置：len - 1。
- 循环条件：i < j。
- 不要动 \0：\0 是字符串结束符，不参与交换。

## 13.5 考试手写代码“救命”技巧

1. 头文件：用到 printf 写 #include <stdio.h>，用到 malloc 写 #include <stdlib.h>，用到 strlen 写 #include <string.h>，用到 sqrt 写 #include <math.h>。
2. 变量初始化：计数器 count、累加器 sum 一定要初始化为 0。
3. 分号检查：if、for、while 后面千万别手顺加分号！
4. 返回值：main 函数最后记得写 return 0;。

## 14 C 语言易混淆概念辨析

- 赋值 vs 相等：
  - = 是赋值运算符（把右边的值给左边）。
  - == 是关系运算符（判断左右两边是否相等）。
  - 陷阱：if(x=5) 永远为真，x 变为 5；if(x==5) 才是判断。

- **字符 vs 字符串:**
  - 'a': 字符常量, 占 1 字节。
  - "a": 字符串常量, 包含 'a' 和 '\0', 占 2 字节。
- **逻辑与 vs 按位与:**
  - && (逻辑与): 短路特性, 结果为 0 或 1。
  - & (按位与): 按二进制位操作, 如  $3 \& 5$  ( $011 \& 101 = 001$ )。
- **sizeof vs strlen:**
  - sizeof: 运算符, 编译时计算, 统计类型或变量占用的内存字节数 (包含 \0)。
  - strlen: 函数, 运行时计算, 统计字符串的有效字符长度 (遇到 \0 停止, 不含 \0)。
- **指针数组 vs 数组指针:**
  - int \*p[10]: 数组, 里面存了 10 个 int\* 指针。
  - int (\*p)[10]: 指针, 指向一个包含 10 个 int 的数组。
- **p++ vs (\*p)++:**
  - p++: 指针向后移动一个单位 (地址改变)。
  - (\*p)++: 指针指向的值加 1 (地址不变, 内容改变)。
- **break vs continue:**
  - break: 彻底跳出当前循环 (或 switch)。
  - continue: 跳过本次循环剩余语句, 直接进入下一次循环判断。
- **形参 vs 实参:**
  - 单向传递: 实参的值传给形参, 形参改变不影响实参 (除非传指针)。

## 15 冷门与进阶考点拾遗

---

- **逗号运算符:**
  - 优先级最低。
  - 表达式的值是最后一个表达式的值。
  - 例:  $a = 3, 4, 5$ ;  $\rightarrow a$  为 3 (赋值优先级高于逗号)。
  - 例:  $a = (3, 4, 5)$ ;  $\rightarrow a$  为 5。
- **宏定义陷阱:**
  - 宏只是简单的文本替换, 不进行计算。
  - 例: #define S(r) r\*r。若调用 S(a+b), 展开为  $a+b*a+b$ , 而非  $(a+b)*(a+b)$ 。
  - 对策: 宏定义中每个参数都要加括号, 整体也要加括号。
- **转义字符 (Escape Characters):**

- 详见 附录 4：字符与字符串陷阱深度解析。
- 核心考点：\ddd (八进制) vs \xhh (十六进制) 的长度限制与非法值判断。
- 格式化输出细节：
  - %5d: 域宽为 5, 右对齐。
  - %-5d: 域宽为 5, 左对齐。
  - %05d: 域宽为 5, 不足补 0。
  - %5.2f: 总宽 5, 小数点后 2 位。
- 运算符优先级口诀：
  - 单目 (!, ++, -, sizeof) > 算术 (\*, /, %) > 算术 (+, -) > 移位 («, ») > 关系 (>, <) > 相等 (==, !=) > 位逻辑 (&, ^, |) > 逻辑 (&&, ||) > 三目 (? :) > 赋值 (=) > 逗号 (,)。
  - 记不住就加括号!
- typedef 与 #define 的区别：
  - #define 是预处理指令, 仅做文本替换。
  - typedef 是编译指令, 为类型创建别名, 有作用域检查。
  - 例: typedef int\* ptr; ptr a, b; → a, b 都是指针。
  - 例: #define PTR int\* PTR a, b; → a 是指针, b 是整型 (替换后为 int \* a, b;)。

## 16 全卷易错点与陷阱大盘点

本章节汇总了全卷最容易丢分的陷阱, 建议考前反复查阅!

### 16.1 语法与逻辑陷阱

- 除法运算：
  - 5 / 2 结果为 2 (整数除法截断小数)。
  - 5.0 / 2 结果为 2.5 (浮点数参与运算)。
- 赋值与相等：
  - 致命错误: if (a = 5)。这会将 5 赋值给 a, 且表达式结果为真 (非 0), 导致条件永远成立。
  - 正确写法: if (a == 5)。

```
1 int a = 5 / 2;    // a = 2
2 double b = 5.0 / 2; // b = 2.5
```

```
1 if (a = 5) { ... } // 错误! 永远为真, a 被改为 5
```

```
2 if (a == 5) { ... } // 正确
```

- 浮点数比较:

- 禁止: `if (f == 0.0)` (因精度问题可能不准确)。
- 正确: `if (fabs(f) < 1e-6)`。

```
1 if (f == 0.0) ... // 危险!
2 if (fabs(f) < 1e-6) ... // 安全
```

- 逻辑短路:

- `A || B`: 若 A 为真, B 不执行。
- `A && B`: 若 A 为假, B 不执行。
- 真题: `k = (n=1) || (m=2);` → m 仍为原值, 不会被赋值为 2。

```
1 int n = 1, m = 0, k;
2 k = (n = 1) || (m = 2);
3 // n=1, k=1, m=0 (m=2 未执行)
```

- 自增自减:

- `b = a++`: 先用 a 的旧值赋给 b, a 再自增。
- `b = ++a`: a 先自增, 再把新值赋给 b。

```
1 b = a++; // b=a, a=a+1
2 b = ++a; // a=a+1, b=a
```

- 宏定义陷阱:

- 宏只是简单的文本替换, 不自动加括号。
- `#define S(x) x*x` → `S(a+b)` 变为 `a+b*a+b` (错误)。
- 正确: `#define S(x) ((x)*(x))`。

```
1 #define S(x) x*x
2 S(a+b) -> a+b*a+b // 错误
3 #define S(x) ((x)*(x))
4 S(a+b) -> ((a+b)*(a+b)) // 正确
```

## 16.2 数组与指针陷阱

- 字符串长度:

- `sizeof("abc")` = 4 (包含末尾的 `'\0'`)。
- `strlen("abc")` = 3 (不包含 `'\0'`)。

```
1 char s[] = "abc";
2 sizeof(s); // 4 (''\0')
3 strlen(s); // 3
```

- 数组越界:

- 定义 `int a[10];`, 有效下标是 0 到 9。访问 `a[10]` 是越界!

```
1 int a[10];  
2 a[10] = 5; // 越界! 最大下标是 9
```

- 指针类型辨析:

- `const` 修饰 (口诀: 左定值, 右定向):

- \* `const int *p: *p` (内容) 不可变, `p` (指向) 可变。

- \* `int * const p: p` (指向) 不可变, `*p` (内容) 可变。

- 指针数组 vs 数组指针:

- \* `int *p[10]`: 数组, 存了 10 个指针。

- \* `int (*p)[10]`: 指针, 指向长度为 10 的数组。

- 函数指针:

- \* 定义: `int (*func)(int, int);`

- \* 用途: 回调函数 (如 `qsort` 的比较函数)。

- 详见附录 [复杂指针类型代码详解](#)。

- 野指针:

- 定义指针时未初始化: `int *p;` → `p` 指向随机地址, 直接 `*p = 10;` 会崩溃。

- 正确: `int *p = NULL;` 或 `int a; int *p = &a;`。

```
1 int *p; *p = 10; // 崩溃! p 未初始化  
2 int *p = NULL; // 安全
```

## 16.3 控制结构与输入输出陷阱

- Switch 语句四大雷区 (真题解析):

- 雷区 1: Case 标签必须是常量

- \* 错误: `case c1:` (变量)。

- \* 正确: `case 1:` 或 `case 'A':` (整型/字符型常量表达式)。

- 雷区 2: Case 标签必须是整型

- \* 错误: `case 1.0f:` (浮点数)。

- \* 正确: `switch` 表达式和 `case` 标签都必须是整型 (`int`, `char`, `enum`)。

- 雷区 3: 语法结构错误

- \* 错误: `switch(a);` (多了分号, 导致 `switch` 无效)。

- \* 错误: `switch a` (少了括号)。

- 雷区 4: 穿透性 (Fallthrough)

\* case 语句块结束后若没有 break，会直接执行下一个 case 的内容。

```

1 // 典型真题错误示例：
2 switch (a) {
3     case c1: ... // 错！变量
4     case 1.5: ... // 错！浮点
5 }
6 switch (a); { ... } // 错！分号截断
7

```

#### • 悬挂 Else:

- else 总是与最近的未匹配的 if 配对，与缩进无关。

```

1 if (a)
2     if (b) x++;
3 else y++; // 这个 else 属于 if(b)

```

#### • 分号错误:

- if (x > 0); → 分号表示空语句，if 实际上什么也没控制。

```

1 if (x > 0); // 多了分号
2     x++;    // 无论 x 是否 > 0 都会执行

```

#### • scanf 缓冲区残留:

- 先输入数字再输入字符时，回车符会被 %c 读取。
- 解决: scanf(" %c", &ch); (%c 前加空格吃掉空白符)。

```

1 scanf("%d", &n);
2 scanf(" %c", &ch); // 空格吃掉回车

```

## 16.4 函数参数传递陷阱

#### • 修改实参需传指针:

- 原理: C 语言函数参数默认是值传递 (Pass by Value)。形参只是实参的拷贝，修改形参不会影响实参。
- 正确做法: 传地址 (&var)，用指针接收 (\*ptr)，解引用修改 (\*ptr = val)。
- 常见错误: 直接传值，在函数内修改形参，以为实参也会变。

```

1 // 错误：值传递，a, b 没变
2 void swap_wrong(int x, int y) { int t=x; x=y; y=t; }
3 // 正确：地址传递
4 void swap_right(int *x, int *y) { int t=*x; *x=*y; *y=t; }
5 // 调用：swap_right(&a, &b);

```

#### • 数组传参退化为指针:

- 原理：数组名作为函数参数传递时，会退化为指向首元素的指针。
- 后果：在函数内部无法通过 `sizeof(arr)` 获取数组总大小（只能得到指针大小，4 或 8 字节）。
- 正确做法：必须额外传递数组长度 `n`。

```
1 void func(int arr[]) {  
2     // sizeof(arr) 在这里等于 sizeof(int*)，即 4 或 8  
3     // 无法知道数组真实长度！  
4 }  
5 // 正确：void func(int arr[], int n)
```

## 17 简答题预测汇总

以下是根据历年考点和课程核心内容整理的简答题题库，建议背诵关键词。

### 17.1 计算机基础与算法

Q1: 简述冯·诺依曼体系结构的核心思想（“存储程序”原理）。

A: 将程序（指令序列）和数据一起存储在计算机的存储器中，计算机在运行时自动地从存储器中取出指令并执行，而不需要人工干预。

Q2: 什么是算法？它有哪些基本特征？

A: 算法是解决特定问题的一系列明确、有限的指令。五大特征：

1. 有穷性：步骤有限，在有限时间内完成。
2. 确定性：每一步指令清晰无歧义。
3. 可行性：每一步都能通过基本运算实现。
4. 输入：0 个或多个输入。
5. 输出：1 个或多个输出。

Q3: 结构化程序设计的三种基本控制结构是什么？

A: 顺序结构、选择结构（分支结构）、循环结构。

### 17.2 C 语言核心概念

Q4: 什么是指针？为什么要使用指针？

A: 指针是存储内存地址的变量。作用：

1. 可以直接操作内存地址，提高程序效率。
2. 实现函数间的大数据传递（避免拷贝整个数组或结构体）。



3. 支持动态内存分配 (malloc/free)。
4. 处理复杂的数据结构 (如链表、树)。

**Q5: 简述递归函数的优缺点。**

**A: 优点:** 代码简洁, 逻辑清晰, 适合解决具有重复子结构的问题 (如汉诺塔、树的遍历)。**缺点:** 函数调用开销大, 占用栈空间多, 深度过深可能导致栈溢出, 效率通常低于迭代。

**Q6: 结构体 (struct) 和共用体 (union) 的区别是什么?**

**A: 结构体:** 每个成员拥有独立的内存空间, 总大小大于等于所有成员大小之和 (受内存对齐影响)。所有成员可以同时存在。**共用体:** 所有成员共享同一块内存空间, 总大小等于最大成员的大小。同一时刻只能存储其中一个成员的值。

**Q7: 局部变量和全局变量的区别?**

**A: 作用域:** 局部变量仅在定义它的函数或代码块内有效; 全局变量在整个源程序文件中有效。**生命周期:** 局部变量随函数调用创建, 函数结束销毁 (static 除外); 全局变量在程序运行期间一直存在。**优先级:** 同名时, 局部变量屏蔽全局变量。

**Q8: 关键字 static 的作用有哪些?**

**A:**

1. **修饰局部变量:** 改变生命周期, 使其在程序运行期间一直存在, 但作用域不变 (仍局限于函数内)。
2. **修饰全局变量/函数:** 限制作用域, 使其仅在当前源文件中可见, 不能被其他文件引用 (隐藏)。

## 17.3 常见算法特点与比较

**Q9: 简述三种基本排序算法 (冒泡、选择、插入) 的区别与特点。**

**A:**

- **冒泡排序:** 两两比较相邻元素, 大数下沉。
  - 最好  $O(n)$  (已有序), 最坏  $O(n^2)$ 。
  - 稳定。
- **选择排序:** 每次从未排序区选最小的放到已排序区末尾。
  - 无论是否有序, 复杂度恒为  $O(n^2)$ 。
  - 不稳定 (如序列 5, 8, 5, 2, 第一趟交换第一个 5 和 2, 导致两个 5 相对位置改变)。
- **插入排序:** 将新元素插入到已排序序列的合适位置。

- 最好  $O(n)$ ，最坏  $O(n^2)$ 。对于**基本有序**的数据效率最高。
- 稳定。

Q10: 顺序查找与二分查找的对比。

A:

- 顺序查找：
  - 适用性：无序或有序数组均可，链表也可。
  - 复杂度： $O(n)$ 。
- 二分查找：
  - 适用性：**必须是有序数组**（支持随机访问）。
  - 复杂度： $O(\log_2 n)$ 。效率远高于顺序查找。

Q11: 什么是算法的时间复杂度和空间复杂度？

A:

- 时间复杂度：算法执行所需时间与问题规模  $n$  之间的增长关系（通常用大  $O$  表示法，如  $O(n), O(n^2)$ ）。衡量算法运行快慢。
- 空间复杂度：算法执行过程中所需的存储空间与问题规模  $n$  之间的增长关系。衡量算法占用内存多少。

## 17.4 实验课补充简答题

Q12: 有序数组与无序数组在操作效率上有何区别？

A:

- 无序数组：插入最快  $O(1)$ （直接追加），查找和删除较慢  $O(n)$ （线性查找）。
- 有序数组：查找最快  $O(\log n)$ （二分查找），插入和删除较慢  $O(n)$ （需要移动元素保持有序）。

Q13: 栈 (Stack) 的主要特性和基本操作有哪些？

A:

- 特性：后进先出 (LIFO, Last In First Out)。
- 基本操作：
  - Push (入栈)：将元素放入栈顶。
  - Pop (出栈)：移除并返回栈顶元素。
  - Peek (查看栈顶)：返回栈顶元素但不移除。
  - IsEmpty (判空)：检查栈是否为空。

**Q14: 如何利用栈实现括号匹配（如 JSON 字符串校验）？**

**A:** 遍历字符串，遇到左括号（如 {, [）入栈；遇到右括号（如 }, ]）时，检查栈顶元素是否为对应的左括号。若是，则弹出栈顶元素继续；若否或栈为空，则匹配失败。遍历结束后栈应为空。

**Q15: 在“找出兴趣值最大的 k 个客户”问题中，为什么不需要全排序？**

**A:** 全排序的时间复杂度通常为  $O(n \log n)$  或  $O(n^2)$ 。如果只需前  $k$  个最大值，可以使用部分选择排序（执行  $k$  轮选择），时间复杂度为  $O(k \cdot n)$ 。当  $k \ll n$  时，效率更高。

**Q16: 递归解决汉诺塔问题的核心逻辑是什么？**

**A:** 将  $n$  个盘子从 A 移到 C 的过程分解为三步：

1. 将  $n - 1$  个盘子从 A 移到 B（借助 C）。
2. 将第  $n$  个盘子（最大的）从 A 移到 C。
3. 将  $n - 1$  个盘子从 B 移到 C（借助 A）。

这是一个典型的分治递归思想。

**Q17: 计算机五大部件（运算器、控制器、存储器、输入设备、输出设备）是什么？**

**A:** 这五大部件构成了冯·诺依曼体系结构的基础。

- **运算器：**负责算术和逻辑运算。
- **控制器：**指挥各部件协调工作。
- **存储器：**存放程序和数据。
- **输入/输出设备：**负责与外部交互。

详细的原理介绍及结构图请跳转至 [附录：冯·诺依曼体系结构详解](#)。

## A 冯·诺依曼体系结构详解

---

计算机硬件系统主要由五大部件组成：运算器、控制器、存储器、输入设备和输出设备。这种结构被称为冯·诺依曼体系结构。

### 1. 运算器 (Arithmetic Logic Unit, ALU):

- **原理：**计算机的“大脑”之一，负责执行所有的算术运算（如加、减、乘、除）和逻辑运算（如与、或、非、异或）。
- **作用：**对数据进行加工处理。

### 2. 控制器 (Control Unit, CU):

- **原理：**计算机的“指挥中心”。它从存储器中取出指令，进行译码分析，然后向其他部件发出控制信号。
- **作用：**保证计算机各部件有序、协调地工作。通常与运算器集成在一起，统称为**中央处理器 (CPU)**。

### 3. 存储器 (Memory):

- **原理：**具有记忆功能的部件，由许多存储单元组成，每个单元都有唯一的地址。
- **作用：**用于存放程序（指令序列）和数据。分为**内存储器**（速度快，CPU 直接访问）和**外存储器**（容量大，长期保存）。

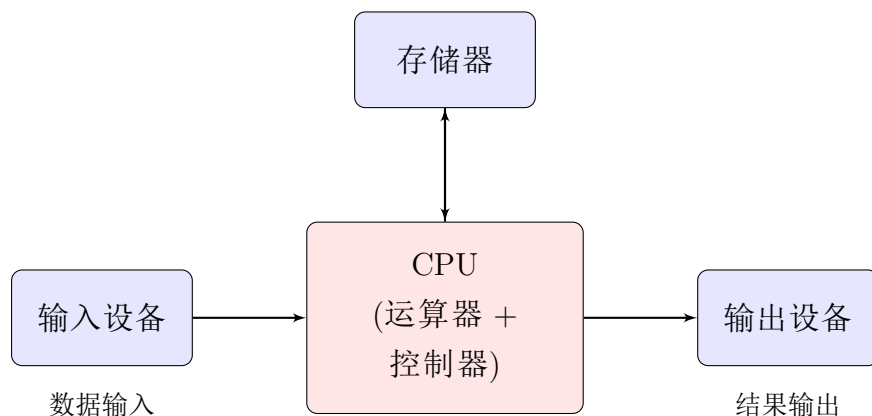
### 4. 输入设备 (Input Device):

- **原理：**将外部信息（文字、声音、图像等）转换为计算机能识别的二进制数据。
- **作用：**向计算机输入原始数据和程序。例如：键盘、鼠标、扫描仪。

### 5. 输出设备 (Output Device):

- **原理：**将计算机处理后的二进制结果转换为人类能识别的形式。
- **作用：**输出处理结果。例如：显示器、打印机、音响。

## 结构示意图



## B 附录 1: 程序填空题常考模式代码详解

### B.1 数学公式实现

- 海伦公式求三角形面积

```
1 #include <math.h>
2 // ...
3 double a, b, c, s, area;
4 // 输入 a, b, c
5 s = (a + b + c) / 2.0;
6 area = sqrt(s * (s - a) * (s - b) * (s - c));
7
```

- 泰勒级数求  $\sin(x)$

```
1 #include <math.h>
2 // ...
3 double x, term, sum;
4 int n = 1;
5 // 输入 x
6 term = x;
7 sum = x;
8 // 循环终止条件: 某一项绝对值小于 1e-5
9 while (fabs(term) >= 1e-5) {
10     term = -term * x * x / ((n + 1) * (n + 2));
11     sum += term;
12     n += 2;
13 }
14
```

### B.2 最大公约数 (辗转相除法)

```
1 int a, b, temp;
2 // 输入 a, b
3 while (b != 0) {
4     temp = a % b;
5     a = b;
6     b = temp;
7 }
8 // 循环结束后, a 即为最大公约数
```

### B.3 条件判断 (三目运算符嵌套)

```
1 // 求 a, b, c 中的最大值
2 int max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
```

## C 附录 2: 标准算法模板

### C.1 排序算法

- 冒泡排序 (Bubble Sort)

```
1 void bubble_sort(int arr[], int n) {
2     int i, j, temp;
3     for (i = 0; i < n - 1; i++) {
4         for (j = 0; j < n - 1 - i; j++) {
5             if (arr[j] > arr[j + 1]) {
6                 temp = arr[j];
7                 arr[j] = arr[j + 1];
8                 arr[j + 1] = temp;
9             }
10        }
11    }
12 }
13
```

- 选择排序 (Selection Sort)

```
1 void selection_sort(int arr[], int n) {
2     int i, j, min_idx, temp;
3     for (i = 0; i < n - 1; i++) {
4         min_idx = i;
5         for (j = i + 1; j < n; j++) {
6             if (arr[j] < arr[min_idx]) min_idx = j;
7         }
8         if (min_idx != i) {
9             temp = arr[i];
10            arr[i] = arr[min_idx];
11            arr[min_idx] = temp;
12        }
13    }
14 }
15
```

## C.2 查找算法

- 二分查找 (Binary Search)

```
1 int binary_search(int arr[], int n, int key) {  
2     int low = 0, high = n - 1, mid;  
3     while (low <= high) {  
4         mid = low + (high - low) / 2;  
5         if (arr[mid] == key) return mid;  
6         else if (arr[mid] < key) low = mid + 1;  
7         else high = mid - 1;  
8     }  
9     return -1;  
10 }  
11
```

## C.3 其他常用算法

- 素数判断

```
1 int is_prime(int n) {  
2     if (n <= 1) return 0;  
3     int limit = (int)sqrt(n);  
4     for (int i = 2; i <= limit; i++) {  
5         if (n % i == 0) return 0;  
6     }  
7     return 1;  
8 }  
9
```

- 字符串逆序

```
1 void reverse_string(char str[]) {  
2     int len = strlen(str);  
3     int i = 0, j = len - 1;  
4     char temp;  
5     while (i < j) {  
6         temp = str[i]; str[i] = str[j]; str[j] = temp;  
7         i++; j--;  
8     }  
9 }  
10
```

## D 附录 3: 复杂指针类型深度解析

## D.1 1. const 与指针的爱恨情仇

核心原理: const 修饰的是紧跟在它后面的那个“东西”。

- 常量指针 (Pointer to Constant): `const int *p`
  - 解析: const 修饰的是 int (内容)。
  - 理解: \*p 被锁死了, 不能通过 p 修改它指向的数据。但 p 自己没锁, 可以改指别人。
  - 场景: 函数参数 `void func(const char *str)`, 保证函数内部不会修改传入的字符串。
- 指针常量 (Constant Pointer): `int * const p`
  - 解析: const 修饰的是 p (指针变量本身)。
  - 理解: p 被锁死了, 一旦初始化就不能指向别处。但 \*p (指向的内容) 是自由的。
  - 场景: 硬件驱动中固定地址的寄存器指针。

```

1 int a = 10, b = 20;
2 const int *p1 = &a; // "我只读, 不改内容"
3 // *p1 = 30; // Error: read-only location
4 p1 = &b;      // OK: p1 可以变心
5
6 int * const p2 = &a; // "我专一, 只指你"
7 *p2 = 30;      // OK: 内容可以变
8 // p2 = &b;   // Error: assignment of read-only variable

```

核心区别对比表:

| 特性     | <code>const int *p</code> | <code>int * const p</code> |
|--------|---------------------------|----------------------------|
| 名称     | 常量指针                      | 指针常量                       |
| 被锁定的对象 | 内容 (*p)                   | 指针本身 (p)                   |
| 可变的对象  | 指针本身 (p)                  | 内容 (*p)                    |
| 口诀     | 左定值                       | 右定向                        |

## D.2 2. 指针数组 vs 数组指针

核心原理: 优先级 `[] > *`。

- 指针数组: `int *p[10]`
  - 解析: p 先和 [10] 结合 → p 是个数组。数组里存什么? 存 int \*。
  - 内存模型: 占  $10 \times 4/8$  字节 (取决于指针大小)。



- 场景: 字符串数组 `char *names[] = {"Tom", "Jerry"};`, 处理不规则长度的字符串列表。
- 数组指针: `int (*p)[10]`
  - 解析: `(*p)` 强制 `p` 先是因指针。指向什么? 指向 `int[10]` 这种类型的数组。
  - 内存模型: 占 4/8 字节 (只是一个指针)。
  - 场景: 处理二维数组。二维数组名 `arr` 传参时退化为数组指针。

```

1 // 指针数组: 本质是"数组"
2 char *strs[3] = {"Apple", "Banana", "Cherry"};
3 // strs[0] 是一个指针, 指向 "Apple" 的首地址
4
5 // 数组指针: 本质是"指针"
6 int matrix[2][3] = {{1,2,3}, {4,5,6}};
7 int (*p)[3] = matrix; // p 指向 matrix 的第 0 行 (包含3个int的数组)
8 // 访问 matrix[1][2] 的写法:
9 // 1. p[1][2]
10 // 2. (*(p+1)+2)

```

核心区别对比表:

| 特性   | <code>int *p[10]</code>   | <code>int (*p)[10]</code> |
|------|---------------------------|---------------------------|
| 名称   | 指针数组                      | 数组指针                      |
| 本质   | 数组                        | 指针                        |
| 存储内容 | 10 个 <code>int*</code> 指针 | 1 个指向数组的地址                |
| 内存占用 | 10 × 指针大小                 | 1 × 指针大小                  |
| 典型用途 | 字符串数组                     | 二维数组传参                    |

### D.3 3. 函数指针与回调机制

核心原理: 函数名在表达式中被解读为该函数的入口地址。

- 定义: `int (*func)(int, int)`
  - `(*func)`: 表明 `func` 是个指针。
  - `(int, int)`: 表明它指向的函数接受两个 `int` 参数。
  - `int`: 表明它指向的函数返回 `int`。
- `qsort` 深度解析:
  - `void *`: 通用指针, 可以接收任何类型的地址, 但不能直接解引用。
  - 类型转换: `*(int*)a`。先将 `void*` 强转为 `int*` (告诉编译器这是个整数地址), 再 `*` 取值。

```

1 // 比较函数: 返回 <0, 0, >0

```

```

2 int compare(const void *a, const void *b) {
3     // 1. (int*)a : 把 void 指针 当作 int 指针 看待
4     // 2. *(int*)a : 取出该地址里的整数值
5     return (*(int*)a - *(int*)b);
6 }
7
8 int main() {
9     int values[] = { 88, 56, 100, 2, 25 };
10    // qsort 内部会多次调用 compare 函数来决定顺序
11    // 这就是"回调": 你把逻辑传给 qsort, qsort 回头调用你的逻辑
12    qsort(values, 5, sizeof(int), compare);
13 }

```

## D.4 4. 二维数组 vs 二维动态数组 (内存模型与访问)

核心原理: 内存连续性决定了访问方式的本质不同。

- 静态二维数组: `int arr[M][N]`
  - 内存布局: 线性连续。所有元素像糖葫芦一样串在一起。
  - 寻址公式:  $\text{Addr} = \text{Base} + (i * N + j) * \text{sizeof}(\text{int})$
  - 过程:
    1. CPU 拿到基地址 Base。
    2. 编译器直接生成计算指令:  $i * N + j$ 。
    3. 一次性算出最终地址, 直接访问。
  - 特点: 速度快, 但列数  $N$  必须在编译时确定 (或作为类型信息存在)。
- 动态二维数组 (二级指针): `int **p`
  - 内存布局: 离散破碎。有一个“脊柱”数组存指针, 指向多根“肋骨”数组存数据。
  - 寻址公式:  $\text{Addr} = *(\text{p} + i) + j * \text{sizeof}(\text{int})$  (伪代码)
  - 过程:
    1. 第一次访存: 访问 `p[i]` (即 `*(p+i)`), 从内存中读取第  $i$  行的首地址 RowAddr。
    2. 第二次访存: 在 RowAddr 的基础上偏移  $j$ , 访问最终数据。
  - 特点: 灵活 (每行长度可以不同), 但多一次内存访问, 且缓存命中率 (Cache Hit) 可能较低。

```

1 // 1. 静态二维数组
2 int arr[3][4];
3 // arr[1][2] -> 编译器直接算出偏移量: 1*4 + 2 = 6 -> 访问第6个元素

```

```

4
5 // 2. 动态二维数组 (二级指针)
6 int **p = (int**)malloc(3 * sizeof(int*)); // 脊柱 (存行指针)
7 for(int i=0; i<3; i++) {
8     p[i] = (int*)malloc(4 * sizeof(int)); // 肋骨 (存数据)
9 }
10 // p[1][2] -> CPU先去 p[1] 查到地址 0x5000, 再去 0x5000 + 2*4 取值

```

核心区别对比表:

| 特性   | int arr[M][N]      | int **p           |
|------|--------------------|-------------------|
| 内存布局 | 连续的一整块             | 破碎的 (指针数组 + 多块数据) |
| 寻址方式 | 一次计算 (基址 + 偏移)     | 两次访问 (查表 + 偏移)    |
| 类型   | int (*) [N] (数组指针) | int ** (二级指针)     |
| 传参写法 | func(int a[] [N])  | func(int **a)     |

## D.5 5. 真题解析: 二维数组指针表示法 (a[i][j] 的 N 种写法)

题目: 定义二维数组 int a[m][n], 以下表示 a[i][j] 的方式正确的是?

- A. \*(a + i\*n + j)
- C. (\*(a + i) + j)

深度辨析:

- 选项 C (正确): (\*(a + i) + j)
  1. a: 类型是 int (\*) [n] (行指针)。
  2. a + i: 指向第 i 行 (地址步长为  $n \times 4$ )。
  3. \*(a + i): 解引用行指针, 得到第 i 行 (即一维数组名 a[i])。此时类型退化为 int \* (列指针)。
  4. \*(a + i) + j: 在第 i 行内偏移 j 个元素。
  5. \*(...): 取出最终的整数值。
- 选项 A & B (错误原因深度剖析):
  - 选项 A: \*(a + i\*n + j)
    - \* 错误本质: 步长错误。
    - \* a 是行指针, a+1 的含义是“下一行”。
    - \* a + (i\*n + j) 意味着跳过了  $i \times n + j$  行! 这会访问到极其遥远的非法内存区域。
    - \* 修正: 必须强转为 int\* 才能按“个”跳。即 \*((int\*)a + i\*n + j)。

- 选项 B:  $*(a + j*n + i)$ 
  - \* 错误本质: 步长错误 + 逻辑混乱。
  - \* 同样未强转, 导致按“行”跳跃。
  - \* 即使强转了, 公式  $j \times n + i$  也是错误的 (这是列优先存储的计算方式, 而 C 语言是行优先存储)。

扩展: 静态数组 vs 动态数组的写法通用性

| 写法                                     | 静态数组 <code>int a[M][N]</code> | 动态数组 <code>int **p</code> |
|----------------------------------------|-------------------------------|---------------------------|
| <code>a[i][j]</code>                   | ✓                             | ✓                         |
| <code>*(*(a+i)+j)</code>               | ✓ (行指针 → 列指针)                 | ✓ (二级指针 → 一级指针)           |
| <code>*[a[i]+j]</code>                 | ✓                             | ✓                         |
| <code>((*[a+i]))[j]</code>             | ✓                             | ✓                         |
| <code>*(&amp;a[0][0] + i*N + j)</code> | ✓ (内存连续)                      | ✗ (内存不连续)                 |

结论: 前四种写法是通用的 (语法糖不同, 底层逻辑不同但表现一致)。唯独线性计算法 (最后一行) 只适用于内存连续的静态数组。

## 附录 4: 字符与字符串陷阱深度解析

### D.6 1. 转义字符 (Escape Characters) 全解

核心定义: 以反斜杠 \ 开头的特殊字符序列。

- 常见转义字符表:

| 转义字符            | 含义                   | ASCII | 备注                        |
|-----------------|----------------------|-------|---------------------------|
| <code>\n</code> | 换行 (Newline)         | 10    | 光标移到下一行开头                 |
| <code>\r</code> | 回车 (Carriage Return) | 13    | 光标移到本行开头                  |
| <code>\t</code> | 水平制表符 (Tab)          | 9     | 光标移到下一个制表位                |
| <code>\b</code> | 退格 (Backspace)       | 8     | 光标左移一格                    |
| <code>\0</code> | 空字符 (Null)           | 0     | 字符串结束标志                   |
| <code>\\</code> | 反斜杠本身                | 92    |                           |
| <code>\'</code> | 单引号                  | 39    | 用于字符常量 <code>'\''</code>  |
| <code>\"</code> | 双引号                  | 34    | 用于字符串常量 <code>"\""</code> |

### D.7 2. 数值转义: `\ddd` vs `\xhh`

核心区别对比表:

| 特性   | \ddd (八进制)   | \xhh (十六进制)   |
|------|--------------|---------------|
| 前缀   | \ 后直接跟数字     | \x            |
| 有效字符 | 0-7          | 0-9, a-f, A-F |
| 长度限制 | 最多 3 位       | 无限制 (贪婪匹配)    |
| 示例   | '\101' ('A') | '\x41' ('A')  |

### 高频陷阱解析:

- 陷阱 1: 非法八进制
  - '\08' 是非法的! 因为 8 不是八进制数。
  - 编译器会报错, 或者将其解析为 '\0' 和 '8' (如果是字符串)。
- 陷阱 2: 八进制截断
  - 题目: `strlen("\128")` 是多少?
  - 解析: \12 是一个八进制转义 (ASCII 10), 8 是普通字符。
  - 结果: 长度为 2。
- 陷阱 3: 十六进制贪婪
  - 题目: `strlen("\x123")` 是多少?
  - 解析: 编译器会一直读下去, 试图解析 0x123 (291), 这超出了 `char` (0-255) 的范围。
  - 结果: 通常报错或发生截断。
  - 对策: 如果要表示 \x12 和字符 3, 必须写成 `"\x12" "3"` (利用字符串自动拼接)。
- 陷阱 4: \x vs 0x
  - `char c = '\x41';` (正确, 字符'A')
  - `int n = 0x41;` (正确, 整数 65)
  - `char c = '\0x41';` (错误! 这是 '\0' + 'x' + ...)